CS331 Project 2 — conversation with Claude

Team T013. Conversation between Suhani and Claude (Cowork), 2–3 November 2026. Suhani's messages and Claude's replies are verbatim. Questions Claude asked through the question tool are shown with their options and the answers given. Tool calls and Claude's internal reasoning are not shown; files Claude wrote are noted in brackets.

SUHANI · 02 NOV 2026, 22:51 IST

Can u look through this repo?
u have seen the project proposal?
https://docs.google.com/document/d/19G069v5OEzf9qigTIG8ZSF2_GKOpRCowl-NK72--e-M/edit?tab=t.0#heading=h.8bo0zw2q2pow

CLAUDE · 02 NOV 2026, 22:52 IST

Yes, I've read both. Short version: the repo has the plan and no work in it yet, and the proposal you actually sent has moved away from that plan in a few places that matter.

What's in the repo (Miloni's, 4 commits, all on 24 Oct):

- README, PROJECT2_PROPOSAL.md, PROJECT2_PLAN.md and PROJECT1_AUDIT.md. Planning docs only. There's no code, no captures, no docs/landscape.md, no manifest.
- Your local copy shows all four files as modified, but that's only Windows line endings. The content hasn't changed, so don't commit it.
- One oddity: PLAN.md says "Nothing here is written into the repository", but it's in the repository.

Where the sent Google Doc differs from the repo version:

- QUIC isn't a depth dive anymore. The repo version had two dives, QUIC and "what a passive tap can see". The doc says "pick 1 or 2 failures" and leaves QUIC as scenario 6 of 9. Sir's one concrete objective was "do QUIC", so this is the gap he's most likely to notice.
- Certificate abridging and capture vantage point were dropped. Sir named both in the meeting. The plan's L3 limitation (vantage point) depends on the dropped scenario, so the plan's matrix and the proposal no longer line up. ECH survived as scenario 8.
- "What it identifies" now says an application (Chrome, Firefox, curl). The plan's L5 argues the opposite: a fingerprint identifies a library and its configuration, and two apps on one library get the same hash. Your own "how often different apps end up with the same hash" metric is set up to show that. Pick one claim before the viva.
- The doc now uses ja4db.com to check the fingerprint against the right app, but the caveat that it's unversioned, community-submitted data is gone. If ja4db match rate is a metric, say what it's weighed against.
- Two additions in the doc are good and aren't in the plan: the "What we will measure" section (parses at all, stability, ja4db match, collision rate) and step 1, diffing your Project 1 tool against FoxIO and tshark. Fold both into the plan.
- The doc says 6–7 limitations; the repo says 7. That's fine.

Schedule: today is D7 on the plan's calendar (2 Nov, QUIC baseline). The D1–D6 work should already exist by now: the landscape table, the baseline agreement diff, the harness, and the L4/L5 captures. None of it is in this repo. If it's sitting in Yash's repo or on someone's laptop, fine. If not, you're a week behind with the mid demo on Thu 5 Nov, and the plan's own 10-day fallback is the realistic path.

Do you want me to update PROJECT2_PLAN.md so its scenario numbers and limitations match the proposal you actually sent?

Sources: objectives_CN_Project (Google Doc), PROJECT2_PLAN.md, PROJECT2_PROPOSAL.md

SUHANI · 02 NOV 2026, 22:58 IST

[Attached file: PROJECT2_PROPOSAL 11.md]

this was the chnaged project proposal!

can we change these files?
make them actually easier to read + follow? there is no implementatio plan as well.
like prof likes to ask the basics of basics and why we don'y that or stuff like that.
whatever we know, we should know fully

[Claude wrote file: BASICS.md]
[Claude wrote file: PROJECT2_PROPOSAL.md]
[Claude wrote file: PROJECT2_PLAN.md]
[Claude wrote file: IMPLEMENTATION.md]

CLAUDE · 02 NOV 2026, 23:04 IST

Now the README.

[Claude wrote file: README.md]

CLAUDE · 02 NOV 2026, 23:04 IST

The QUIC key steps match RFC 9001 Appendix A. Saving into your folder now.

SUHANI · 02 NOV 2026, 23:04 IST

[Attached file: IMPLEMENTATION_PLAN.md]

ohkay, have a look at this implementation plan!
What I want from you: you know the direction of the project we are taking (the main gist of it) forget the repo forget these implentation plan,
in the chat, ask me any number of decisions (instead of you taking it on your own, like for eg. c++ or python) u give me (if req, why this or that, and what u prefer)

prepare me a whole doc like this, and I will tell you what to do.
DO NOT ASSUME ANY DECISION AS YOUR OWN!
Ask away.
can have any number of questions

CLAUDE · 02 NOV 2026, 23:05 IST

Read it. Below is every decision I'd otherwise make myself. Each one lists the options and, where it helps, which I'd pick and why. Answer by number (e.g. "8: Python"), skip any you want to leave open, and add your own options where none fit.

A. Scope

1. How many limitations? Exactly 7 (repo proposal) or 6–7 (the Google Doc you sent)? I'd pick 6–7. The doc sir saw says that, and a weak 7th row is worse than a strong 6th.

2. Depth dives: QUIC + passive visibility, QUIC only, or a different pair? I'd pick QUIC + passive visibility. QUIC is sir's ask. Passive visibility is cheap and drops out of the L2 captures anyway.

3. What counts as the "fix"?

4. (a) a working QUIC extractor that existing tools lack
5. (b) a combined signal (e.g. JA4 + JA4T + something) compared against plain JA4
6. (c) a precise statement of what a fix would need

7. (d) a mix I'd pick (a), with (c) for the second dive.

8. QUIC scope: v1 Initial only, or also v2, Retry, 0-RTT, coalesced packets? I'd pick v1 as the must and coalesced packets as the should. The rest become the "cases we couldn't handle" list.

9. Which fingerprints are in scope? JA3, JA3S, JA4, JA4S, JA4T, JA4X, JA4H. Add JA4L (latency), JARM (active), or JA4SSH, or leave them out? I'd pick the seven and mention JARM in one line as the active kind we don't use.

10. Which scenarios stay? Same app on different OSes, old vs new version, repeated connections, same library, IPv4/IPv6, QUIC, post-quantum (PQ), ECH, encrypted DNS, capture point, certificate abridging. Mark any to cut or demote to write-up only. I'd make abridging write-up only, and Android optional.

11. Does the Project 1 tool take part as a third implementation in comparisons, or is it dropped? The Google Doc's plan step 1 says to compare it. Is that still on?

B. Language and tools

1. Language for the QUIC extractor: Python, C++, Rust, or Go?
2. Python: fastest to write, easiest to explain line by line in a viva, and the `cryptography` package covers HKDF and AES.
3. C++: more control over bytes, good C++ practice. Costs time on CMake, OpenSSL APIs and memory bugs.

4. Rust/Go: only if someone on the team already knows it. I'd pick Python. The marks are for understanding the protocol, not for speed, and sir asks basics. Pick C++ only if the team wants the C++ practice for its own sake.

5. Crypto library: OpenSSL (with C++) or Python `cryptography`? This follows from 8.

6. How do we read pcaps in our own code?

    - scapy: easy, slow
    - dpkt: light
    - pyshark: wraps tshark, so it's not independent of the tool we're testing
    - libpcap (C++)
    - our own minimal pcap reader I'd pick dpkt or our own reader. The pcap format is a 24-byte header plus records, simple to explain.

7. ClientHello parser for the extracted bytes: reuse Project 1's (it was verified against the spec), write a new one, or use tshark's decode? I'd reuse Project 1's parser function only and re-test it.

8. JA4 for QUIC hellos:

    - our own JA4 code
    - reuse Project 1's JA4 code
    - only compare against tshark's QUIC JA4 output I'd reuse Project 1's JA4 and check it field by field against tshark.

9. Which FoxIO implementation: the Python script, the Rust binary, or both? And do we pin a Wireshark version, or use whatever each person has? I'd pick one FoxIO implementation, pin a Wireshark ≥ 4.2, and have everyone use the same one.

10. Add Zeek or Suricata as extra implementations, or keep it to FoxIO + tshark (+ Project 1)? I'd skip them. Setup cost, little new information.

11. Harness language: Python scripts + Makefile, bash only, or Jupyter notebooks? I'd pick Python + Makefile. Notebooks are hard to reproduce and review.

12. Tests: pytest only, or GoogleTest too (only if C++)? CI on GitHub Actions: yes or no?

C. Capture setup

1. Main capture machine:

    - Linux VM on someone's laptop
    - native Linux
    - Windows + Wireshark/Npcap
    - a mix, recorded per capture Who has Linux natively?

2. Targets: public sites (Cloudflare, Google), our own local server (nginx/Caddy with HTTP/3), or both? I'd pick both. The local server gives control (certs, PQ, QUIC), public sites give realism. Study B needs about 10 servers, which has to mean public sites.

3. Client list (pick any): Chrome, Firefox, Edge, curl (OpenSSL), Windows built-in curl (Schannel), python requests, Go net/http, Node, aioquic, quiche, Android Chrome, an Android app, Safari (does anyone have a Mac or iPhone?).
4. Old versions from: Docker images with pinned tags, portable browser builds, distro package archives, or skip old versions entirely?
5. Android: real phone over the laptop hotspot, emulator, or skip?
6. IPv6: does anyone know if the campus network or hostel Wi-Fi gives IPv6? Fallback: phone hotspot, VM-to-VM lab only, or drop the scenario?
7. ECH: real ECH (Firefox + DoH + a Cloudflare test page), our own ECH server, or a labelled simulation? I'd pick real via Firefox, with simulation only if that fails.
8. Proxy for the capture-point test: mitmproxy (decrypts and re-encrypts, so the server sees the proxy's JA4), a CONNECT proxy like squid (no TLS termination), or both? I'd pick both. They show two different effects.
9. Connections per client for stability: 10, 20, or 50?
10. Commit SSLKEYLOGFILE files to the repo? They only decrypt our own lab traffic, but they'd be public if the repo is public.
11. Pcaps in git directly, via Git LFS, or hosted elsewhere and downloaded by a script? Plain git is fine if the corpus stays under about 50 MB. How big do you expect it to get?
12. Is the repo public or private?

D. Evaluation

1. Metrics: the Google Doc's four (parse rate, stability, ja4db match, collisions) plus tool agreement? Add or drop any?
2. ja4db: manual lookups recorded with date, the downloadable dump if one exists, or drop it?
3. Strictness of "every number from `make eval`": strict for every number, or only for the limitation table and metrics?

E. Team and process

1. Who is P1–P4? Names and roles (QUIC / TLS visibility / clients / capture+harness), or a different split?
2. Which repo is the real one: Miloni's CS331-Phase2, Yash's TLS_fingerprint_CN, or a new one?
3. How strict are branches, PRs and reviews: every change via a reviewed PR, or PRs only for code?
4. AI logs: what format does the course require (raw export per session, one file per person)?

5. What's actually done so far from the old Days 1–6? Landscape table, captures, anything?

6. Deadlines: is mid demo 5/6 Nov and freeze 12 Nov still right? Did sir give any official dates?

F. The document itself

1. Format: a .md in the repo (like this one), a shareable doc, or both?

2. Tone: formal and rule-heavy like the uploaded plan, or plainer like the rewrites I did earlier?

3. Sections to include? The uploaded plan has these:

    - outcomes
    - language choices
    - repo layout
    - QUIC stages
    - limitation table
    - harness
    - team
    - calendar
    - acceptance criteria
    - viva questions
    - shortcuts to avoid
    - status

    Keep all, drop some, or add any (e.g. a per-scenario recipe section, a theory/basics section)?

4. Name the people in the doc, or keep P1–P4?

SUHANI · 02 NOV 2026, 23:19 IST

1. 6-7
2. QUIC + passive visibility
3. no, we have tp direct;y imlemetn the quic first as given in rfc (no changes) and then find out what is responsible that it does not work practically (so find out where it lacks so we can improve) and then find its fix so it can be implemented

yk what, I think so we need to read it first. can u please make a reading guide for the team? simple terms + very very simple lanaguge

SUHANI · 02 NOV 2026, 23:20 IST

1. 6-7
2. QUIC + passive visibility
3. no, we have tp direct;y imlemetn the quic first as given in rfc (no changes) and then

find out what is responsible that it does not work practically (so find out where it lacks so we can improve) and then find its fix so it can be implemented

yk what, I think so we need to read it first. can u please make a reading guide for the team? simple terms + very very simple lanaguge
Don't start from the very basics of networking, but start from TLS ....we have zero base on QUIC, and that is something we need a proerp grasp over + what decission are we making and why?

CLAUDE, QUESTIONS · 02 NOV 2026, 23:20 IST

Q: What format should the team reading guide be in?

- Markdown in repo — docs/reading-guide.md in CS331-Phase2. Everyone reads it on GitHub, sits next to the code.
- Shareable doc — A Claude Doc with a link you send the team. Downloads as Word/PDF.
- Lesson page — Brilliant-style card-by-card HTML: question first, then explanation. Better for actually learning, worse as a reference.
- Markdown + lesson page — Reference doc in the repo, plus a lesson page for the QUIC part only.

SUHANI, ANSWERS · 02 NOV 2026, 23:20 IST

Your questions have been answered: "What format should the team reading guide be in?"="Markdown in repo". You can now continue with these answers in mind.

[Claude wrote file: reading-guide.md]

CLAUDE · 02 NOV 2026, 23:23 IST

The reading guide is in your CS331-Phase2 folder as `docs/reading-guide.md`. It isn't committed to git yet.

It has six parts:

1. TLS: the handshake, why the ClientHello is visible, records, and the two tools QUIC also uses (HKDF and AES-GCM).
2. A short recap of fingerprints.
3. QUIC from zero, the longest part. It explains why QUIC exists, how its packets carry the TLS handshake, why anyone can decrypt the first packets but nothing after them, and the cases where real traffic gets messy.
4. One plain paragraph on each of the other scenarios.
5. What you've decided so far: implement QUIC exactly as the RFC says, find where it fails on real traffic, then fix that. The section says why that order.
6. The 29 decisions still open. Each says what the choice is and why it matters, with no answer picked.

Each part ends with the exact RFC sections to read and a few check-yourself questions.

I tested the QUIC key steps against the worked example in RFC 9001 Appendix A and they match. The guide points to BASICS.md for more detail on fingerprints. That's one of the files I wrote earlier, and it's in the same folder.

Once you've read it, send me your answers to Part 6 by number and I'll write the implementation plan from them.

SUHANI · 03 NOV 2026, 10:29 IST

okay, we are thinking of diving into two,
one the 6-7 problems and then the QUIC implementation?
should we be doing that>?

also...what all should be including in the 6-7 problems though?
like ask us the basic implementation of how it should be!
like the lnaguage? (c/rust/etc, prof said it's fine to use AI but u need to know what is even in the code? or should we mix?)
ask us proerly this time

CLAUDE · 03 NOV 2026, 10:29 IST

Short answer: yes, do both. But they aren't two separate projects. The QUIC implementation is one of the 6–7 problems (QUIC hides the ClientHello), taken all the way down.

How I'd shape it:

- Track 1 (breadth): the 6–7 problems. Little code: captures, existing tools (FoxIO, tshark), and a few scripts to compare their outputs. This answers "map the landscape".
- Track 2 (depth): QUIC. Build Initial extraction exactly as RFC 9001 says, run it on real traffic, find where it breaks, then fix that. This is the part sir asked for ("do QUIC") and the part that gets marked as your own work.

Two things to know before you answer:

- Passive visibility (JA4X/JA4H needing keys) is cheap. It probably works better as one of the 6–7 rows than as a third deep study. You decide.
- Time is the real constraint. Freeze is 12 Nov and nothing is captured yet. Both tracks fit only if they run in parallel with the team split, not one after the other.

I'll ask in rounds: structure first, then which problems, then how we measure them, then the QUIC code (language, mixing, how AI-written code gets understood).

CLAUDE, QUESTIONS · 03 NOV 2026, 10:29 IST

Q: Overall shape of Project 2?

- Two tracks (Recommended) — Track 1: 6–7 limitations using existing tools (breadth). Track 2: QUIC built per RFC, then its practical failures, then a fix (depth). QUIC is also one of the 6–7 rows.
- QUIC only — Everything is about QUIC. The 6–7 limitations are all practical failures found inside QUIC (multi-packet hello, Retry, v2, coalescing...). Narrower but deeper.
- Limitations only — 6–7 limitations with existing tools, no QUIC build. Least work, but ignores sir's 'do QUIC'.

Q: What happens to 'passive visibility' (JA4X/JA4H need keys, what a tap can see)?

- One of the 6–7 rows (Recommended) — Run captures with and without keys, report the difference. About a day of work.
- Separate deep study — Full study: measure how much JA4X adds over JA4S across ~10 servers, plus a passive/needs-keys/unrecoverable table. 2–3 extra days.
- Drop it — Don't cover it.

Q: How do the two tracks run in time?

- In parallel (Recommended) — Split the team: two people on QUIC from now, two on the limitation captures. Only way both fit before 12 Nov.
- Limitations first — Everyone does the 6–7 rows first, then everyone moves to QUIC. Everyone sees everything, but QUIC gets squeezed.
- QUIC first — Everyone builds QUIC first, then does the limitation rows. Risk: the breadth part gets rushed at the end.

Q: How many limitations do we commit to?

- 6 firm, 7th if evidence (Recommended) — Promise 6. Add a 7th only if it has a real capture. Matches the doc sir saw (6–7).
- Exactly 7 — Commit to 7 rows.
- Whatever evidence supports — No fixed number, report all that have captures.

SUHANI, ANSWERS · 03 NOV 2026, 10:31 IST

Your questions have been answered: "Overall shape of Project 2?"="Two tracks (Recommended)", "What happens to 'passive visibility' (JA4X/JA4H need keys, what a tap can see)?"="One of the 6–7 rows (Recommended)", "How do the two tracks run in time?"="Limitations first", "How many limitations do we commit to?"="6 firm, 7th if evidence (Recommended)". You can now continue with these answers in mind.

CLAUDE, QUESTIONS · 03 NOV 2026, 10:31 IST

Q: Which transport/stack problems go in the list? (pick any)

- QUIC hides the hello — Tools built for TCP miss the ClientHello inside encrypted QUIC Initials. This is the row that leads into Track 2.
- Post-quantum big hello — ML-KEM key share adds ~1.2 KB, hello spans 2 TCP packets / 2 QUIC Initials. Easy with curl + OpenSSL 3.5 (--curves).
- Capture point matters — Proxy, NAT/VPN, NIC offloads change what the capture sees (JA4T, segment sizes, whose hello it is). Needs a proxy setup.

- IPv4 vs IPv6 — JA4 should stay same, JA4T changes (MSS 1460 vs 1440). Needs a network with IPv6.

Q: Which identity/stability problems go in the list? (pick any)

- JA3 unstable for Chrome — Extension shuffling: many JA3s over 20 connections, one JA4. Easy and very clear.
- Same library = same hash — Two different apps on one TLS library collapse to one JA4, so a fingerprint doesn't name an app.
- Spoofing (uTLS/curl-impersonate) — A script copies Chrome's hello exactly and gets Chrome's JA4. Tests the core assumption of fingerprinting. Was a gap in Project 1.
- Version/OS drift — Same app, different version or OS gives a different fingerprint, so databases go stale. Old versions are slow to set up.

Q: Which visibility problems go in the list? (pick any)

- JA4X/JA4H need keys — TLS 1.3 encrypts the certificate and HTTP headers. With-keys vs without-keys on the same capture. (The passive-visibility row.)
- ECH hides the real hello — Tap sees only the outer ClientHello and a fake hostname. Testable with Firefox + DoH + Cloudflare.
- Encrypted DNS — No fingerprint changes, but the domain name you'd pair with it is gone. An attribution analysis, not a tool failure.
- Certificate abridging — Drafts replace the certificate with a hash/proof; JA4X has nothing to read. Not deployed, so write-up only.

Q: If more than 7 get picked, how do we cut?

- Merge related ones — e.g. ECH + encrypted DNS = one 'hostname disappears' row; JA3 instability + same-library = one 'stability and collisions' row.
- Keep the easiest to evidence — Drop whatever needs the hardest setup (IPv6, Android, ECH) first.
- Keep the ones sir named — Prioritise QUIC, PQ, IPv6, ECH, encrypted DNS, abridging, GRO/capture point.
- We'll decide after captures — Try more, keep the 6–7 with the strongest evidence.

SUHANI, ANSWERS · 03 NOV 2026, 10:47 IST

The user answered: "Which transport/stack problems go in the list? (pick any)"="IPv4 vs IPv6,QUIC hides the hello,Capture point matters", "Which identity/stability problems go in the list? (pick any)"="JA3 unstable for Chrome,Same library = same hash,Spoofing (uTLS/curl-impersonate)", "Which visibility problems go in the list? (pick any)"="- *JA4X/JA4H need keys — select, with a TLS 1.3 qualification.* Without secrets, a passive observer cannot read the TLS 1.3 Certificate message or HTTP headers. With secrets, JA4H still requires decoding the relevant HTTP protocol and its header compression. Don’t say JA4X always needs keys: certificates in TLS 1.2 handshakes can be visible without them. See [RFC 8446 §2](https://www.rfc-editor.org/rfc/rfc8446.html#section-2), [§4.4.2](https://www.rfc-editor.org/rfc/rfc8446.html#section-4.4.2), [RFC 9113](https://www.rfc-editor.org/rfc/rfc9113.html), and [RFC 9204](https://www.rfc-editor.org/rfc/rfc9204.html). - *ECH hides the real hello — select, but change “fake

hostname.”* A passive tap sees the outer ClientHello; ECH protects the inner ClientHello, including the real SNI. The outer hello may use a *public name—it isn’t necessarily a “fake hostname.” Confirm the client actually *accepted ECH; merely seeing an ECH extension does not prove that. See [RFC 9849](https://www.rfc-editor.org/rfc/rfc9849.html). - *Encrypted DNS — select if attribution analysis is in scope.* It hides the DNS query from a passive observer, but the domain is not necessarily “gone”: SNI may still reveal it unless ECH is in use, and IP addresses can provide clues. This does not inherently change the TLS fingerprint. See [RFC 8484](https://www.rfc-editor.org/rfc/rfc8484.html) and [RFC 9250](https://www.rfc-editor.org/rfc/rfc9250.html). no need for the fourth one", "If more than 7 get picked, how do we cut?"="Keep the ones sir named". Read the answers carefully — they may request clarification, changes, or that you not proceed — and follow what they actually say.

CLAUDE, QUESTIONS · 03 NOV 2026, 10:48 IST

Q: You picked 9; keeping sir's named ones gives QUIC, IPv6, capture point, ECH, encrypted DNS (5). Is encrypted-DNS attribution in scope, and how?

- Merge with ECH (Recommended) — One row: 'hostname attribution' — what a tap can still learn about the destination with ECH on/off and DNS plain/encrypted (SNI, public name, DNS, IP clues).
- Separate row — ECH and encrypted DNS as two rows.
- Drop encrypted DNS — Attribution analysis out of scope; keep ECH only.

Q: Which of the non-sir-named picks fill the remaining 1–3 slots? (pick any)

- JA4X/JA4H need keys (TLS 1.3) — The passive-visibility row you chose earlier. With/without secrets on the same capture, plus a TLS 1.2 contrast.
- Stability + collisions merged — JA3 unstable for Chrome and same-library-same-hash as one row: 'what a fingerprint can and can't identify'.
- Spoofing — uTLS or curl-impersonate produces Chrome's exact JA4 from a script.
- JA3 and same-library separate — Keep them as two rows instead of merging.

Q: Which tools compute fingerprints in the limitation experiments?

- FoxIO JA4+ — The reference implementation of the JA4 family.
- Wireshark/tshark — Independent second implementation (JA3, JA3S, JA4; also decrypts QUIC Initials).
- Our Project 1 tool — Third opinion; the Google Doc plan step 1 said to compare it.
- Zeek or Suricata — What real IDS deployments run. Extra setup.

Q: What do clients connect to during captures?

- Both (Recommended) — Own local server for controlled tests (QUIC settings, certs, TLS 1.2 vs 1.3); public sites (Cloudflare, Google) for realism and ECH.
- Public sites only — Simplest; less control. ECH and QUIC are easy on Cloudflare.
- Own server only — Full control and repeatable, but ECH and real-world behaviour are harder.

SUHANI, ANSWERS · 03 NOV 2026, 10:56 IST

Your questions have been answered: "You picked 9; keeping sir's named ones gives QUIC, IPv6, capture point, ECH, encrypted DNS (5). Is encrypted-DNS attribution in scope, and how?"="Merge with ECH (Recommended)", "Which of the non-sir-named picks fill the remaining 1–3 slots? (pick any)"="JA4X/JA4H need keys (TLS 1.3),Stability + collisions merged", "Which tools compute fingerprints in the limitation experiments?"="FoxIO JA4+,Wireshark/tshark", "What do clients connect to during captures?"="Both (Recommended)". You can now continue with these answers in mind.

CLAUDE, QUESTIONS · 03 NOV 2026, 10:56 IST

Q: Where do we capture from?

- Linux VM, one shared setup — Everyone captures from the same kind of VM (same tcpdump, offloads, MTU). Most comparable, easiest to document.
- Native Linux laptop — If one of you runs Linux natively: no VM networking in between, real NIC offloads.
- Windows + Wireshark — Captures Windows apps directly (Npcap). Needed anyway for Windows-only clients.
- Mix, recorded per capture — Whatever each person has, with the setup written in each capture's metadata.

Q: Which TCP/TLS clients do we test? (pick any)

- Chrome + Firefox — Two different TLS libraries (BoringSSL vs NSS). Chrome is needed for the JA3 shuffling row.
- curl + Python requests — Both on OpenSSL: the classic same-library pair for the collision row.
- Go + Node — Go has its own TLS library; Node uses OpenSSL. More library variety.
- Edge / Windows curl — Edge = Chromium on Windows; Windows built-in curl uses Schannel, a different library.

Q: Which QUIC clients for Track 2? (need about 4)

- Chrome — Biggest QUIC user; splits and shuffles CRYPTO frames, so it's the hardest real case.
- Firefox — Uses its own QUIC stack (neqo) with NSS.
- curl HTTP/3 — Uses ngtcp2 or quiche depending on build. Needs a curl built with HTTP/3.
- aioquic / quiche client — Python (aioquic) or Rust (quiche) test clients. Fully controllable: can force Retry, v2, big hellos.

Q: How many repeated connections per client for stability?

- 20 (Recommended) — Enough to show Chrome's JA3 spread clearly; still quick to capture.
- 10 — Minimum that's still a measurement.
- 50 — Stronger numbers, more capture time and repo size.

SUHANI, ANSWERS · 03 NOV 2026, 11:00 IST

The user answered: "Where do we capture from?"="We will use the Linux VM, and the implementation will be done by one person only!", "Which TCP/TLS clients do we test? (pick any)"="Chrome + Firefox,curl + Python requests", "Which QUIC clients for Track 2? (need about 4)"="keep all except quiche", "How many repeated connections per client for stability?"="20 (Recommended)". Read the answers carefully — they may request clarification, changes, or that you not proceed — and follow what they actually say.

CLAUDE, QUESTIONS · 03 NOV 2026, 11:01 IST

Q: Language for the QUIC extractor (the Track 2 code one person writes)?

- Python (Recommended) — Shortest code; every step (HKDF, header protection, AES-GCM, frames) is a few readable lines with the `cryptography` package. Easiest for the other three to read and for sir to follow. Slow, which doesn't matter for pcaps.
- C++ — Explicit byte handling and bounds checks, good systems practice. Costs: CMake, OpenSSL's verbose C API, memory bugs. About 2–3x the code for the same steps.
- Rust — Safe byte parsing, good crates (ring/rustls pieces). Only sensible if the implementer already knows Rust.
- C — Closest to the RFC's byte layouts, but everything is manual, including buffer safety. Highest bug risk in the time left.

Q: One language for everything, or mix?

- One language (Recommended) — If the extractor is Python, scripts and analysis are Python too. One toolchain, anyone can run anything.
- Mix: core + Python scripts — Extractor in C++/Rust/C, everything around it (running tools, comparing, stats) in Python. Two toolchains to set up on every machine.

Q: Sir allows AI but you must know what's in the code. How will AI be used for the extractor?

- Human writes, AI explains (Recommended) — Implementer writes each function from the RFC; AI answers questions and reviews. Slowest, but every line is theirs.
- AI drafts, human rewrites — AI drafts a function; implementer rewrites it in their own words, tests it, and comments each step with the RFC section.
- AI writes, human verifies — AI writes the code; implementer checks each function against the RFC and the test vectors and writes a walkthrough. Fastest, weakest in a viva.

Q: With one implementer, how do the other three know the code well enough for the viva?

- Walkthrough doc — Implementer writes a file that maps each function to its RFC section in plain words.
- Live code walkthrough — One session where the implementer explains the code step by step and everyone asks questions.
- Each runs it on their capture — Each teammate runs the extractor on a pcap they captured and explains the output.

- Each owns one step in viva — Each person can explain one stage (header, keys, header protection, frames) in depth.

SUHANI, ANSWERS · 03 NOV 2026, 11:06 IST

The user answered: "Language for the QUIC extractor (the Track 2 code one person writes)?"="C++", "One language for everything, or mix?"="Mix: core + Python scripts", "Sir allows AI but you must know what's in the code. How will AI be used for the extractor?"="no, AI does it all!", "With one implementer, how do the other three know the code well enough for the viva?"="forget that, let's do the main implementation! we have to submit this by Thurs (and the writeup should seem non-AI) are we done?". Read the answers carefully — they may request clarification, changes, or that you not proceed — and follow what they actually say.

CLAUDE, QUESTIONS · 03 NOV 2026, 11:06 IST

Q: What exactly is due Thursday?

- Implementation plan doc — The written plan (this doc) is the submission; code and results come later.
- Plan + working extractor — Plan plus the C++ QUIC extractor running on at least one real capture.
- Everything — Captures for the 6 rows, extractor, results and the write-up, all by Thursday.

Q: How does the C++ extractor read pcap files?

- libpcap (Recommended) — Standard C library behind tcpdump; pcap_open_offline + pcap_next_ex. One dependency, installed everywhere.
- Own pcap reader — Parse the pcap file format ourselves (24-byte header + records). No dependency; pcapng files would need converting first.
- PcapPlusPlus — C++ wrapper with packet parsing built in. Bigger dependency, hides layers we may want to show.

Q: After the extractor gets the ClientHello bytes, who computes JA4?

- C++ computes JA4 itself — Full pipeline in one binary; JA4 checked field by field against tshark's QUIC JA4.
- Python, reuse Project 1 JA4 — C++ dumps the ClientHello bytes; Python reuses Project 1's tested JA4 code. Matches the core + scripts mix.
- Only compare with tshark — We just extract and validate the bytes; JA4 values come from tshark.

Q: What does 'implement exactly per RFC' cover in the first version?

- Client Initial, v1 (Recommended) — Header parse, key derivation, header protection, decrypt, CRYPTO reassembly by offset, ClientHello. Everything else becomes a 'practical failure' to find.
- Client + server Initial — Also decrypt the server's Initial (needs the client's original DCID) to get ServerHello / JA4S over QUIC.

- v1 + coalesced + Retry — Handle coalesced packets and Retry from the start. Fewer things left to 'find', more to build.

SUHANI, ANSWERS · 03 NOV 2026, 11:11 IST

The user answered: "What exactly is due Thursday?"="sorry, it is by Friday! 10 AM. Deadline extended: The Project 2 deadline has been extended from 4th November to Friday, 6th November, 10:00 AM. Submission (same as before): All deliverables (code, report, presentation slides, and AI logs) are submitted via GitHub. Late submissions: We check the GitHub commit timestamp. If you need more time, you may submit after Friday 10:00 AM, but commits after the deadline will incur a 5% penalty on Project 2. GitHub repository link submission: https://forms.gle/myizaAhuutFkCbDY6 Evaluations: Evaluations will be held on Saturday and Sunday, 14th and 15th November, so you have time to prepare your presentation. If your team wants to present before the weekend, we can arrange that, as we did for Project 1. Slots will be updated at the link below soon. You are expected to arrive 30 minutes before your allotted time slot. https://docs.google.com/spreadsheets/d/1xs-PS2L0N6fQY7Z74RBGfxeJ5JyvMDNm8nG3RA8e-W8/edit?pli=1&gid=1884078349#gid=1884078349 Repository structure (same as the Project 1 submission): Plain Text README.md /code → all source code /report → final report (PDF) /ppt → presentation slides (PDF or .pptx) /ai → all AI prompts and conversations ⚠️ Note on the AI folder: Please include all the prompts and conversations you had with AI tools during this project. No summaries, only raw conversations. Missing or incomplete AI logs may lead to a deduction of marks. All the best! TAs, CS 331 + they have shared a final report submission template... writeup can be done letter, u just make the implenation plan for running + keep it as sleek and simple + lightweight as possible", "How does the C++ extractor read pcap files?"="libpcap (Recommended)", "After the extractor gets the ClientHello bytes, who computes JA4?"="Python, reuse Project 1 JA4", "What does 'implement exactly per RFC' cover in the first version?"="Client Initial, v1 (Recommended)". Read the answers carefully — they may request clarification, changes, or that you not proceed — and follow what they actually say.

[Claude wrote file: IMPLEMENTATION_PLAN.md]

CLAUDE · 03 NOV 2026, 11:13 IST

The implementation plan is in your CS331-Phase2 folder as `IMPLEMENTATION_PLAN.md`. It's built from your answers and runs from today to the Friday 10 AM deadline.

It has six parts:

1. Setup commands for the Linux VM.
2. One standard way to take every capture.
3. The six limitations, each with what to capture, what to expect and the RFC sections:
4. QUIC hides the ClientHello
5. IPv4 vs IPv6
6. capture point
7. stability and collisions

8. JA4X/JA4H need keys on TLS 1.3
9. ECH and encrypted DNS as one hostname row
10. The C++ extractor in seven stages, tested against RFC 9001 Appendix A, then the practical-failures run and the fix.
11. Three Python scripts.
12. A day-by-day schedule and a submission checklist that matches the TA folder layout.

I filled in four small choices you weren't asked about. Change any of them:

- OpenSSL 3 for the crypto, since you chose C++.
- A plain Makefile instead of CMake, to keep it light.
- Caddy as the local TLS 1.3 / HTTP/3 server, and `openssl s_server` for the TLS 1.2 comparison.
- mitmproxy for the proxy test.

I also changed the ECH references in BASICS.md and PROJECT2_PLAN.md to RFC 9849 and made the "ECH must be accepted" correction. PROJECT2_PLAN.md and IMPLEMENTATION.md are now out of date; IMPLEMENTATION_PLAN.md is the one to follow.

Two things you should hear before Friday:

- On the write-up seeming non-AI: the TAs want every raw AI conversation in `ai/`, and incomplete logs lose marks. That includes this one. Write the report plainly in your own voice, but don't trim the logs to make the work look less AI-made. That would cost more than it saves.
- With C++, AI writing all of it and one implementer, the viva is where you're exposed. Evaluation is 14–15 Nov, so the whole team has about a week after submitting to learn the code. Part 3 of the reading guide and the stage list in the plan are what to use for that.